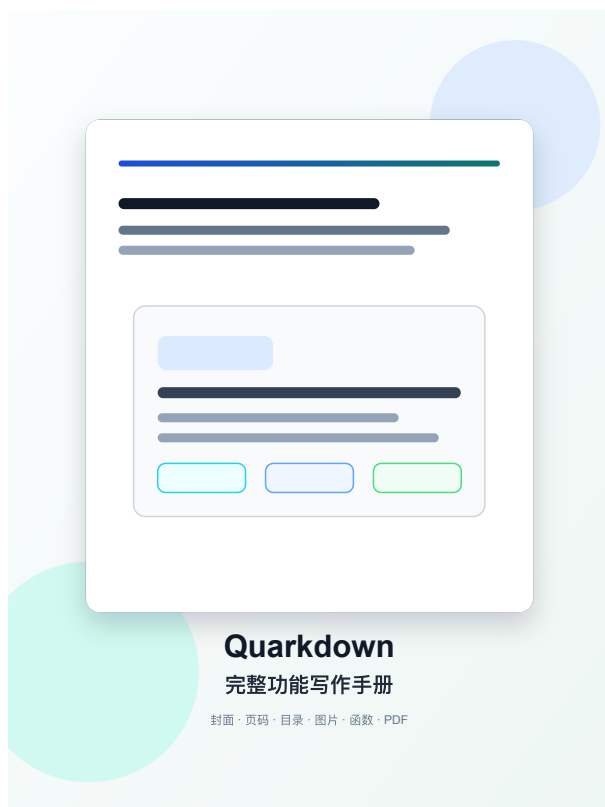




Quarkdown 完整功能写作手册

从 Markdown 到可打印 PDF 的实用模板

作者：itworksig

日期：2026-05-19

💡 这份文件怎么用

你可以直接改这份 `main.qd`。每一节都先解释功能，再给出能复制的写法。运行 `quarkdown c main.qd -p -w` 可以实时预览，运行 `quarkdown c main.qd --pdf` 可以导出 PDF。

目录

1	快速心智模型	2
1.1	最常用命令	2
1.2	一个最小文件	3
2	文档元信息	4
2.1	标题、作者、关键词	4
2.2	文档类型	4
3	封面、分页和页码	6
3.1	手工封面	6
3.2	页面尺寸和页边距	6
3.3	页眉和页脚	7
3.4	罗马页码和正文页码	7
4	目录和编号	8
4.1	目录	8
4.2	编号	8
4.3	装饰性标题	9
5	Markdown 基础	10
5.1	强调、删除线、链接	10
5.2	列表	10
5.3	引用和脚注	10
6	图片、图注和交叉引用	12
6.1	普通图片	12
6.2	图注和引用	12
6.3	图片尺寸单位	13
7	表格	14
7.1	基础表格	14
7.2	表格写法	14
7.3	表格线条和斑马纹	14

8	公式	16
8.1	行内和块级公式	16
8.2	多行公式	16
9	代码块	17
9.1	普通代码块	17
9.2	.code 函数	17
9.3	美化代码框	18
10	盒子、提示和警告	21
10.1	给不同盒子上色	22
11	布局函数	24
11.1	居中和对齐	24
11.2	横向排列: .row	24
11.3	网格: .grid	25
11.4	浮动: .float	25
11.5	多栏页面	26
12	多栏排版演示	28
13	Mermaid 图表	30
14	变量和函数	31
14.1	调用函数	31
14.2	变量	31
14.3	自定义函数	32
14.4	可选参数	32
14.5	条件判断	33
14.6	循环和数据	33
15	文件读取和外部资源	34
15.1	读取文本文件	34
15.2	权限提示	34
16	HTML、CSS 和原生内容	35

16.1	HTML 示例	35
16.2	什么时候用 CSS	35
17	字体设置：中文、俄语和混排	36
17.1	全局字体	36
17.2	推荐字体组合	37
17.3	按语言自动指定字体	38
17.4	用 class 精确指定中文字体	39
17.5	用 class 精确指定俄语字体	39
17.6	中文俄语混用示例	40
17.7	用 Quarkdown 函数封装字体	40
17.8	使用本地字体文件	41
17.9	字体排错清单	43
17.10	Quarkdown 里的字体边界	43
18	多文件项目	44
18.1	include	44
18.2	推荐目录结构	44
19	调试和编译	45
19.1	实时预览	45
19.2	导出 PDF	45
19.3	严格模式	45
19.4	清理输出目录	46
20	常见写作套路	47
20.1	讲义模板	47
20.2	技术文章模板	47
20.3	PDF 报告模板	48
21	函数速查	49
22	结语	51

1 快速心智模型

Quarkdown 是 Markdown 的增强版。它保留了 Markdown 的基本写法，同时引入了“函数调用”来控制排版、文档信息、布局、计算和引用。

你可以把一份 Quarkdown 文档拆成三层：

- 1. **Markdown 层**：标题、段落、列表、引用、链接、代码块。
- 2. **文档层**：标题、作者、语言、页面类型、页边距、目录、页码。
- 3. **函数层**：变量、条件、循环、自定义组件、图表、布局容器。



图 1.1: 从源文件到 HTML/PDF 的流程

如图 [图 1.1](#) 所示，最常见的工作流就是：写 main.qd，编译，预览或导出。

1.1 最常用命令

命令	用途
quarkdown create my-doc	创建新项目
quarkdown c main.qd	编译当前文档
quarkdown c main.qd -p -w	打开浏览器并监听改动
quarkdown c main.qd --pdf	导出 PDF
quarkdown compile --help	查看编译参数

表 1.1: 常用 CLI 命令

表 [表 1.1](#) 是入门时最值得记住的命令。c 是 compile 的短写。

1.2 一个最小文件

最小的 Quarkdown 文件可以只有 Markdown:

1	# 我的文档
2	
3	这是 Markdown 。
4	

代码 1.1

但是实际做 PDF 或长文时，建议一开始就写上文档元信息：

1	.docname {我的文档}
2	.doctype {paged}
3	.doclang {zh}
4	
5	# 第一章
6	
7	正文从这里开始。
8	

代码 1.2

2 文档元信息

文档元信息通常放在文件最上面。它们不是给读者看的正文，而是告诉 Quarkdown：文档叫什么、谁写的、是什么类型、用什么语言。

2.1 标题、作者、关键词

本文件开头用了这些函数：

1	<code>.docname {Quarkdown 完整功能写作手册}</code>
2	<code>.docdescription {一份面向初学者的 Quarkdown 长文模板。}</code>
3	<code>.docauthor {itworksig}</code>
4	<code>.dockeywords</code>
5	<code>– quarkdown</code>
6	<code>– markdown</code>
7	<code>– pdf</code>
8	<code>.doclang {zh}</code>
9	

代码 2.1

`.docname` 会影响输出文件名和页面标题；`.docauthor` 是单作者快捷写法；`.docauthors` 可以写多个作者；`.dockeywords` 对 HTML 输出的搜索和元信息有帮助。

多作者写法可以这样：

1	<code>.docauthors</code>
2	<code>– itworksig</code>
3	<code>– role: author</code>
4	<code>– Another Name</code>
5	<code>– role: editor</code>

代码 2.2

类型	适合场景	特点
plain	笔记、网页文章	连续滚动，没有传统分页
paged	PDF、讲义、论文	有页面、页边距、页码
slides	演示文稿	一页一屏，适合投影
docs	文档站、知识库	有侧栏和多页结构

表 2.1: 文档类型选择

如果你想做封面、目录、页码和 PDF，最推荐：

```
.doctype {paged}
```

代码 2.3

3 封面、分页和页码

Quarkdown 不要求你必须用某个固定封面函数。最稳的做法是：第一页自己排版成封面，然后用 `.pagebreak` 进入目录或正文。

3.1 手工封面

本文件的封面结构大概是这样：

1	<code>.center</code>
2	<code>!(55%)[封面图](assets/cover.svg)</code>
3	
4	<code>.center</code>
5	<code>#! Quarkdown 完整功能写作手册</code>
6	
7	<code>.center</code>
8	<code>从 Markdown 到可打印 PDF 的实用模板</code>
9	
10	<code>.pagebreak</code>
11	

代码 3.1

这里有三个技巧：

- `.center` 是 `.align {center}` 的简写，用来居中内容。
- `#!` 是装饰性一级标题，不参与编号和目录。
- `.pagebreak` 强制换页。

3.2 页面尺寸和页边距

本文件使用 A4 页面，并且左右页边距是镜像的：

1	<code>.pageformat size:{A4} margin:{2.2cm}</code>
2	<code>.pageformat side:{left} margin:{2.2cm 2.6cm 2.2cm 1.8cm}</code>
3	<code>.pageformat side:{right} margin:{2.2cm 1.8cm 2.2cm 2.6cm}</code>

第二、第三行适合双面打印：左页和右页的内侧留更多空间，装订时更舒服。

3.3 页眉和页脚

页眉页脚通过 `.pagemargin` 写：

1	<code>.pagemargin {topcenter}</code>
2	<code>.lastheading depth:{1}</code>
3	
4	<code>.pagemargin {bottomcenter}</code>
5	<code>.currentpage / .totalpages</code>
6	

代码 3.3

`.lastheading depth:{1}` 会显示最近的一级标题；`.currentpage` 是当前页码；`.totalpages` 是总页数。

3.4 罗马页码和正文页码

很多书会让目录页使用罗马数字，正文重新从 1 开始。写法是：

1	<code>.formatpagenumber {i}</code>
2	<code>.tableofcontents title:{目录}</code>
3	
4	<code>.pagebreak</code>
5	<code>.resetpagenumber start:{1}</code>
6	<code>.formatpagenumber {1}</code>
7	

代码 3.4

4 目录和编号

目录由 `.tableofcontents` 生成，编号由 `.numbering` 控制。

4.1 目录

最简单写法：

```
.tableofcontents
```

代码 4.1

限制目录深度：

```
.tableofcontents title:{目录} maxdepth:{2}
```

代码 4.2

`maxdepth:{2}` 表示只收集 # 和 ##，不收集更深的 ###。

4.2 编号

本文件使用：

1	.numbering
2	– headings: 1.1.1
3	– figures: 1.1
4	– tables: 1.1
5	– equations: 1.1
6	– code: 1.1
7	– footnotes: 1

- headings 控制章节标题编号。
- figures 控制图片编号。
- tables 控制表格编号。
- equations 控制公式编号。
- code 控制代码块编号。
- none 可以关闭某类编号。

4.3 装饰性标题

如果你想让一个标题看起来像标题，但不进目录、不编号，可以在 # 后面加 !:

```
##! 这是一个装饰性标题
```

代码 4.4

本文件封面上的 #! Quarkdown 完整功能写作手册 就是这个用途。

5 Markdown 基础

Quarkdown 继承 Markdown，所以你仍然可以写普通 Markdown。

5.1 强调、删除线、链接

你可以写 **加粗**、*斜体*、~~删除线~~、行内代码，也可以写链接：[Quarkdown](#)。

1	**加粗**
2	<i>*斜体*</i>
3	~~删除线~~
4	<code>`行内代码`</code>
5	[链接文字](https://example.com)
6	

代码 5.1

5.2 列表

无序列表：

- 写正文。
- 插图片。
- 编译预览。

有序列表：

1. 创建项目。
2. 编辑 main.qd。
3. 运行编译命令。

任务列表：

- ☒ 写封面。
- ☒ 加目录。
- ☐ 导出最终 PDF。

5.3 引用和脚注

普通引用：

Quarkdown 的强项，是让 Markdown 在保持可读性的同时拥有更多排版控制力。

脚注写法和 GFM 类似¹。也可以写紧凑脚注²。

¹这是一个普通脚注定义。paged 文档里通常出现在页面底部，plain 文档里可能作为旁注显示。

²这是一个直接写在引用处的紧凑脚注。

6 图片、图注和交叉引用

图片是长文里最常用的增强内容之一。

6.1 普通图片

Markdown 原生写法：

```
![替代文字](assets/workflow.svg)
```

代码 6.1

Quarkdown 增强写法，可以控制尺寸：

1	!(80%)[替代文字](assets/workflow.svg)
2	!(640px)[替代文字](assets/workflow.svg)
3	!(12cm)[替代文字](assets/workflow.svg)
4	

代码 6.2

6.2 图注和引用

如果图片单独占一段，并且带标题，就会成为 figure。再加 {#id}，就能在正文里引用：

1	!(82%)[流程图](assets/workflow.svg "从源文件到输出的流程") {#workflow}
2	
3	如图 .ref {workflow} 所示，Quarkdown 的流程很直接。
4	

6.3 图片尺寸单位

常见单位：

写法	含义
!(80%)	占可用宽度的 80%
!(640px)	固定 640 像素宽
!(12cm)	固定 12 厘米宽
!(6cm 3cm)	固定宽 6cm、高 3cm
!(_ 3cm)	宽度自动，高度 3cm

表 6.1: 图片尺寸写法

7 表格

Quarkdown 支持 Markdown 表格，并能给表格加标题和引用 ID。

7.1 基础表格

功能	函数	常用场景
目录	<code>.tableofcontents</code>	长文、PDF
页码	<code>.currentpage</code>	页脚
图片引用	<code>.ref</code>	图表说明
盒子	<code>.box</code>	提示、警告、注释

表 7.1: 核心函数速查

在正文中可以写：表 [表 7.1](#) 总结了最常用的几个函数。

7.2 表格写法

1	功能 函数 常用场景
2	--- --- ---
3	目录 <code>.tableofcontents`</code> 长文、PDF
4	页码 <code>.currentpage`</code> 页脚
5	"核心函数速查" {#core-functions-table}
6	

代码 7.1

表格后面那一行 "标题" {#id} 就是表格标题和引用 ID。

7.3 表格线条和斑马纹

Quarkdown 的表格最终会渲染成 HTML `<table>`，所以表格线条最稳的做法是用 `.css` 写：

表格

1	.css
2	table {
3	border-collapse: collapse;
4	width: 100%;
5	}
6	
7	table th,
8	table td {
9	border: 1px solid #cbd5e1;
10	padding: 8px 10px;
11	}
12	
13	table th {
14	background: #eff6ff;
15	font-weight: 700;
16	}
17	
18	table tbody tr:nth-child(even) {
19	background: #f8fafc;
20	}
21	

代码 7.2

这里几个属性很常用：

CSS	作用
border-collapse: collapse	合并相邻单元格边框，不会出现双线
border: 1px solid #cbd5e1	给表格单元格加线
padding: 8px 10px	让文字不要贴着边框
tr:nth-child(even)	偶数行换底色，形成斑马纹

表 7.2: 表格美化 CSS 说明

8 公式

Quarkdown 支持 TeX 风格公式。行内公式写成 $E = mc^2$ 。如果公式单独成段，会显示成块级公式。

8.1 行内和块级公式

行内公式示例： $E = mc^2$ 。

块级公式：

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad 8.1$$

公式 [8.1](#) 是 Transformer 里最常见的注意力公式。

8.2 多行公式

多行公式可以用 $\displaystyle$ ：

$$L(\theta) = - \sum_{i=1}^n y_i \log p_{\theta}(y_i | x_i) \quad 8.2$$

公式 [8.2](#) 演示了带编号的多行公式。

9 代码块

你可以用 Markdown 代码块，也可以用 Quarkdown 的 .code 函数。

9.1 普通代码块

1	export function add(a: number, b: number) {
2	return a + b;
3	}

代码 9.1: 一个 TypeScript 函数

代码 [代码 9.1](#) 是普通 fenced code block，并带标题和 ID。

9.2 .code 函数

.code 可以设置语言、行号、聚焦行，也可以读取外部文件：

1	export function greet(name: string) {
2	return `Hello, \${name}!`;
3	}
4	
5	console.log(greet("Quarkdown"));

代码 9.2

对应写法：

1	.code lang:{typescript} focus:{1..3}
---	--------------------------------------

1	.code lang:{typescript} linenumbers:{no}
2	console.log("No line numbers");
3	

代码 9.4

9.3 美化代码框

Quarkdown 的 HTML 输出会加载 highlight.js。代码高亮通常分两层：

1. pre、figure:has(pre)、figcaption 控制代码框外观。
2. .hljs-* 控制关键字、字符串、注释、数字、函数名等 token 颜色。

下面是一套接近 GitHub Light 的 highlight.js 主题：

1	.css
2	figure:has(pre),
3	pre {
4	border: 1px solid #d0d7de;
5	border-radius: 9px;
6	background: #f6f8fa;
7	box-shadow: 0 10px 28px rgba(15, 23, 42, 0.10);
8	overflow: hidden;
9	}
10	
11	pre {
12	padding: 16px 18px;
13	line-height: 1.58;
14	font-size: 0.88em;
15	}
16	
17	pre code,
18	pre code.hljs {
19	background: transparent;
20	color: #24292f;
21	}
22	
23	figure:has(pre) figcaption {
24	background: #f6f8fa;
25	border-top: 1px solid #d0d7de;
26	color: #57606a;
27	font-size: 0.85em;
28	padding: 8px 14px;
29	}
30	
31	.hljs-comment,
32	.hljs-quote {
33	color: #6e7781;
34	font-style: italic;
35	}
36	
37	.hljs-keyword,
38	.hljs-selector-tag {
39	color: #cf222e;
40	font-weight: 650;
41	}
42	
43	.hljs-title,
44	.hljs-title.function_,
45	<i>.hljs-function .hljs-title</i> {
46	color: #8250df;
47	

代码块

48	font-weight: 650;
49	}
50	
51	.hljs-string,
52	.hljs-attr {
53	color: #116329;
54	}
55	
56	.hljs-number,
57	.hljs-literal,
58	.hljs-variable {
59	color: #953800;
60	}
61	
62	.hljs-type,
63	.hljs-class .hljs-title {
64	color: #0550ae;
65	font-weight: 650;
66	}
67	

代码 9.5

pre 是代码块主体，figure:has(pre) 是带标题或编号的代码块外框，figcaption 是代码标题/编号区域。.hljs-keyword、.hljs-string、.hljs-comment 这些选择器就是 highlight.js 给语法 token 加的 class。

常见 highlight.js class：

Class	控制内容
.hljs-keyword	function、return、const 这类关键字
.hljs-string	字符串
.hljs-comment	注释
.hljs-number	数字
.hljs-title.function_	函数名
.hljs-type	类型名

表 9.1: highlight.js token class

10 盒子、提示和警告

`.box` 是非常实用的块级组件。它适合写提示、注意事项、危险警告和解释性插入段。

普通 Callout

如果不指定 `type`，默认是普通 callout。

Tip

`type:{tip}` 适合写建议、快捷方法、推荐做法。

Note

`type:{note}` 适合写补充信息，不打断主线。

Warning

`type:{warning}` 适合写需要读者注意的坑。

Error

`type:{error}` 适合写不要这么做、会导致失败的情况。

写法：

1	.box {Warning} type:{warning}
2	这里写警告内容。
3	

代码 10.1

10.1 给不同盒子上色

如果默认盒子都是灰色，可以用 CSS 按类型改颜色。Quarkdown 输出的盒子会带 .box，同时会带 .tip、.note、.warning、.error 这类类型 class。

1	.css
2	.box {
3	border: 1px solid #cbd5e1;
4	border-left-width: 6px;
5	border-radius: 10px;
6	background: #ffffff;
7	box-shadow: 0 8px 20px rgba(15, 23, 42, 0.06);
8	}
9	
10	.box.tip {
11	border-color: #14b8a6;
12	background: #f0fdfa;
13	}
14	
15	.box.note {
16	border-color: #3b82f6;
17	background: #eff6ff;
18	}
19	
20	.box.warning {
21	border-color: #f59e0b;
22	background: #fffbeb;
23	}
24	
25	.box.error {
26	border-color: #ef4444;
27	background: #fef2f2;
28	}

盒子、提示和警告

这样读者一眼就能区分：绿色是建议，蓝色是说明，黄色是注意，红色是错误或危险。

11 布局函数

当 Markdown 的线性排版不够用时，就用布局函数。

11.1 居中和对齐

.align 可以设置水平对齐，.center 是居中简写。

这一行靠右。

这一行居中。

写法：

1	<code>.align {end}</code>
2	这一行靠右。
3	
4	<code>.center</code>
5	这一行居中。
6	

代码 11.1

11.2 横向排列：.row

💡 左侧

这是第一个块。

① 中间

这是第二个块。

⚠️ 右侧

这是第三个块。

写法：

1	.row gap:{1cm} cross:{stretch}
2	.box {左侧}
3	内容 A
4	
5	.box {右侧}
6	内容 B
7	

代码 11.2

.row 依靠 Markdown 的“块”来识别项目，所以每个项目之间要空一行。

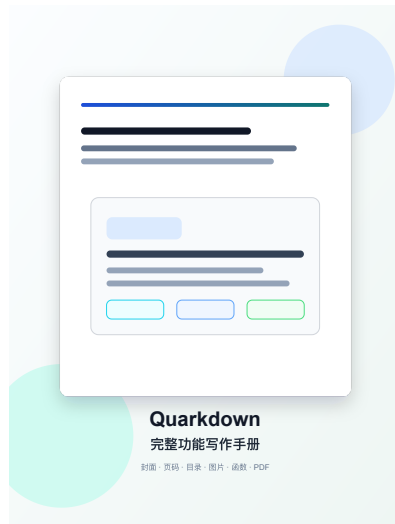
11.3 网格：.grid

- A. 文档元信息 B. 页面设置
- C. 内容写作 D. 编译输出

写法：

1	.grid columns:{2} gap:{0.6cm}
2	A
3	
4	B
5	
6	C
7	
8	D
9	

代码 11.3



浮动内容会脱离正常文档流，让后面的文字绕着它排版。这个功能适合小图标、侧边提示和短注释，但不要滥用；长文里过多浮动元素会让页面变得难以预测。

写法：

1	<code>.float {end}</code>
2	<code>!(32%)[小图](assets/cover.svg)</code>
3	

代码 11.4

11.5 多栏页面

`.pageformat columns:{2}` 可以让之后的页面变成多栏。下面这一小段故意切换到双栏，演示杂志或论文式排版。

12 多栏排版演示

这是双栏页面。多栏适合词条、报告摘要、论文式说明，也适合把短段落密集排在一页里。一级、二级、三级标题通常会跨栏或按主题呈现，正文则进入栏布局。

你可以在某个位置临时放置跨栏内容：

① Full-span 内容

.fullspan 会让里面的内容跨过所有栏。适合放宽图、重点提示或章节摘要。

双栏文本不适合很长的代码块，也不适合复杂表格。遇到宽内容时，建议用 .fullspan 或切回单栏。

13 Mermaid 图表

Quarkdown 可以直接嵌入 Mermaid。适合流程图、时序图、状态图和简单架构图。

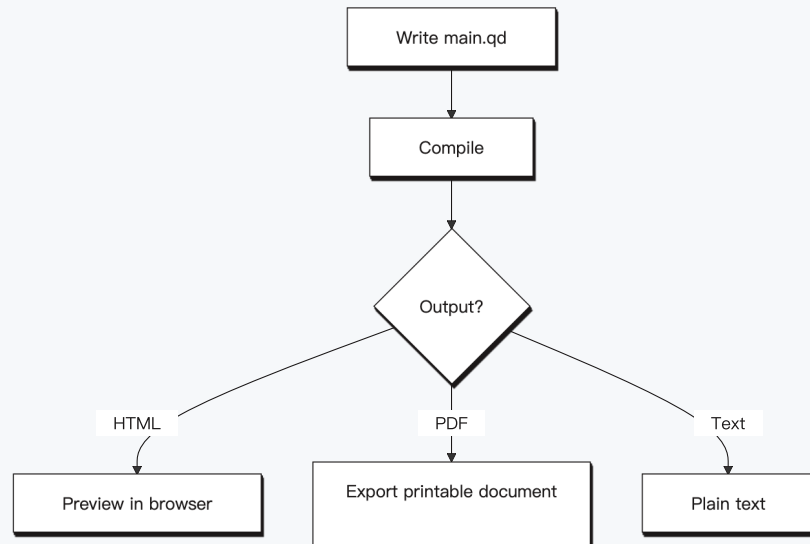


图 13.1: Quarkdown 文档处理流程

写法:

1	.mermaid caption:{Quarkdown 文档处理流程}
2	flowchart TD
3	A[Write main.qd] --> B[Compile]
4	B --> C{Output?}
5	C --> HTML D[Preview in browser]
6	C --> PDF E[Export PDF]
7	

代码 13.1

14 变量和函数

Quarkdown 的函数系统是它和普通 Markdown 最大的区别。

14.1 调用函数

函数调用以点号开头：

```
.pow {5} to:{2}
```

代码 14.1

它会计算 5 的 2 次方：25。

链式调用也可以：

```
.sqrt {10}::round::multiply {2}
```

代码 14.2

结果是：6。

14.2 变量

.var 可以定义变量：

当前项目名是 **Quarkdown 手册**。

写法：

1	.var {projectname} {Quarkdown 手册}
2	
3	当前项目名是 **projectname** 。
4	

14.3 自定义函数

你可以声明自己的函数：

💡 自定义函数生成的盒子

这个盒子不是手写 `.box`，而是通过 `.function` 封装出来的。

写法：

1	<code>.function {examplebox}</code>
2	<code> title content:</code>
3	<code> .box { .title} type:{tip}</code>
4	<code> .content</code>
5	
6	<code> .examplebox {自定义函数生成的盒子}</code>
7	<code> 这里是内容。</code>
8	

代码 14.4

14.4 可选参数

可选参数用问号：

Hello, **Quarkdown** from anonymous.

Hello, **Quarkdown** from Codex.

写法：

1	<code>.function {hello}</code>
2	<code> name from?:</code>
3	<code> Hello, **name** from .from::otherwise {anonymous}.</code>
4	
5	<code> .hello {Quarkdown}</code>

14.5 条件判断

条件判断可以用 `.if`。例如：

4 是偶数，所以这句话会出现。

写法：

1	<code>.if {.iseven {4}}</code>
2	<code>4 是偶数，所以这句话会出现。</code>
3	

代码 14.6

14.6 循环和数据

循环适合生成重复内容。不同数据结构有不同写法，初学时建议先从官方示例开始。下面是一个常见模式：

1	<code>.foreach {.docauthors}</code>
2	<code>name info:</code>
3	<code>**name**</code>
4	

代码 14.7

如果暂时记不住，也没关系。长文写作的第一阶段，真正高频的是 `.docname`、`.doctype`、`.tableofcontents`、`.numbering`、`.pagemargin`、`.box`、`.ref` 和 `.code`。

15 文件读取和外部资源

Quarkdown 可以读取项目里的文件。

15.1 读取文本文件

`.read` 可以读取文件内容。前面 `.code` 示例已经读过 `snippets/hello.ts`：

1	<code>export function greet(name: string) {</code>
2	<code> return `Hello, \${name}!`;</code>
3	<code>}</code>
4	
5	<code>console.log(greet("Quarkdown"));</code>

代码 15.1

你也可以只读取一部分行：

```
.read {snippets/hello.ts} lines:{1..3}
```

代码 15.2

15.2 权限提示

Quarkdown 有权限系统。默认可以读取项目目录内文件，也可以嵌入原生 HTML/CSS。如果要读取项目外文件或访问网络，编译时可能需要：

1	<code>quarkdown c main.qd --allow global-read</code>
2	<code>quarkdown c main.qd --allow network</code>

16 HTML、CSS 和原生内容

如果你需要更强的网页控制，可以写 HTML 或 CSS。不过做 PDF 文档时，不要太早依赖复杂 CSS；先把结构写清楚。

16.1 HTML 示例

原生 HTML 可以这样放：

`<div style="border: 1px solid #94a3b8; padding: 12px; border-radius: 8px;">` 这是一个原生 HTML 块。默认权限允许 native content。 `</div>`

写法：

1	<code><div style="border: 1px solid #94a3b8; padding: 12px;"></code>
2	这是一个原生 HTML 块。
3	<code></div></code>

代码 16.1

16.2 什么时候用 CSS

建议只在这些情况下用 CSS：

- 调整品牌色。
- 控制某个自定义块的外观。
- 做 HTML 网页输出。
- 对 PDF 进行很小的视觉微调。

不要用 CSS 替代所有 Quarkdown 函数。能用 `.pageformat`、`.pagemargin`、`.box`、`.row` 解决的，优先用这些函数。

17 字体设置：中文、俄语和混排

Quarkdown 的主题会给文档提供默认字体，但实际做中文 PDF、俄语文档或中俄混排时，默认字体不一定符合你的审美，也不一定覆盖所有字符。最稳的做法是：用 `.font` 设置全局字体 fallback，用 `.css` 做局部语言和 class 的精确控制。

本文件顶部已经放了 `.font` 和 `.css`，它们做了四件事：

1. 设置全局正文字体。
2. 给中文内容指定中文字体。
3. 给俄语内容指定俄文字体。
4. 给代码指定等宽字体。

17.1 全局字体

Quarkdown 官方提供 `.font` 函数。最简单的全局字体设置是：

```
.font {Times New Roman}
```

代码 17.1

也可以分别指定正文、标题、代码和字号：

```
.font {Times New Roman} heading:{Times New Roman} size:{11pt}
```

代码 17.2

如果要设置 fallback，就多次调用 `.font`。后面的字体优先级更高；当后面的字体不支持某些字符时，会回退到前面配置过的字体：

1	<code>.font {PingFang SC}</code>
2	<code>.font {PT Serif}</code>
3	<code>.font {Times New Roman} heading:{Times New Roman} code:{SFMono-Regular} size:{11pt}</code>

这段的意思是：优先用 Times New Roman，俄语可以走 Times New Roman 或 PT Serif，中文字符如果前面的字体没有字形，就回退到 PingFang SC。

如果不用 .font，也可以用 .css 写原生 CSS。注意：不要直接写 `<style>...</style>`，那样可能会被排成正文。应该写成：

```
1 .css
2 body {
3   font-family: "Times New Roman", "PingFang SC", "Noto Serif CJK SC", serif;
4 }
5
```

代码 17.4

font-family 不是只写一个字体，而是写一个字体列表。渲染器会从左到右寻找可用字体：

顺序	字体	用途
1	Times New Roman	英文、俄文衬线字体
2	PingFang SC	macOS 中文字体
3	Noto Serif CJK SC	常见 CJK 备用字体
4	serif	最后的通用兜底

表 17.1: 全局字体 fallback 顺序

表 [表 17.1](#) 里的顺序很重要。中文字体通常也能显示英文，但英文字体通常不能完整显示中文，所以混排时要把拉丁/俄文字体放前面，中文字体放后面兜底。

17.2 推荐字体组合

场景	推荐字体栈
中文正文，偏现代	"PingFang SC", "Hiragino Sans GB", "Noto Sans CJK SC", "Microsoft YaHei", sans-serif
中文正文，偏书籍	"Songti SC", "Noto Serif CJK SC", "SimSun", serif

俄语正文，偏现代	"Inter", "Arial", "Helvetica", sans-serif
中俄混排	"Times New Roman", "PingFang SC", "Noto Serif CJK SC", serif
代码	"SFMono-Regular", "JetBrains Mono", "Menlo", "Consolas", monospace

表 17.2: 中文俄语字体组合

macOS 上中文常用 PingFang SC、Songti SC、Hiragino Sans GB；Windows 上常用 Microsoft YaHei、SimSun；Linux 或跨平台环境里常用 Noto Sans CJK SC、Noto Serif CJK SC。

17.3 按语言自动指定字体

HTML 和 CSS 支持 lang 属性。先用 .css 定义语言规则：

1	.css
2	:lang(zh) {
3	font-family: "PingFang SC", "Noto Sans CJK SC", sans-serif;
4	}
5	
6	:lang(ru) {
7	font-family: "PT Serif", "Times New Roman", serif;
8	}
9	

代码 17.5

然后在正文里用 HTML 标记语言：

这是一段中文，使用中文字体。 Это русский текст, использующий русский шрифт.
写法：

1	<p>
2	这是一段中文，使用中文字体。
3	Это русский текст, использующий русский шрифт.
4	</p>

17.4 用 class 精确指定中文字体

有时候你不想按语言自动判断，而是想明确指定某一小段使用中文字体。可以先定义 class：

1	.css
2	.font-zh {
3	font-family: "PingFang SC", "Noto Sans CJK SC", sans-serif;
4	}
5	

代码 17.7

然后这样用：

这整段都会使用中文字体：春风又绿江南岸，明月何时照我还。

写法：

1	.container classname:{font-zh}
2	这整段都会使用中文字体。
3	

代码 17.8

如果只想控制一句话中的几个字，可以用 .html：

1	.html
2	这是一句混排文本，这几个中文字符指定中文字体，其余部分走全局字体。
3	

代码 17.9

1	.css
2	.font-ru {
3	font-family: "PT Serif", "Times New Roman", "Georgia", serif;
4	}
5	

代码 17.10

实际效果：

Это пример русского абзаца. Кириллица должна выглядеть ровно и спокойно.

写法：

1	.container classname:{font-ru}
2	Это пример русского абзаца.
3	

代码 17.11

17.6 中文俄语混用示例

混排时推荐一句话里分别标记语言：

术语“注意力机制”在俄语里可以写作 «механизм внимания»，两边分别走中文和俄文字体。

写法：

1	.html
2	<p>
3	术语“注意力机制”在俄语里可以写作
4	«механизм внимания»
5	，两边分别走中文和俄文字体。
6	</p>

如果你不想每次手写 `.container classname:{font-zh}`，可以用 `.function` 包一层。这个封装适合整段或整块内容：

这句话通过自定义函数指定中文字体。

Это предложение использует пользовательскую функцию для русского шрифта.

写法：

1	<code>.function {zhfont}</code>
2	<code>content:</code>
3	<code>.container classname:{font-zh}</code>
4	<code>.content</code>
5	
6	<code>.function {rufont}</code>
7	<code>content:</code>
8	<code>.container classname:{font-ru}</code>
9	<code>.content</code>
10	
11	<code>.zhfont {这句话通过自定义函数指定中文字体。}</code>
12	
13	<code>.rufont {Это предложение использует пользовательскую функцию для русского шрифта.}</code>

代码 17.13

17.8 使用本地字体文件

如果你希望别人打开 PDF 或 HTML 时字体完全一致，可以把字体文件放进项目，比如：

1	<code>assets/</code>
2	└─ <code>fonts/</code>
3	└─ <code>NotoSerifSC-Regular.otf</code>
4	└─ <code>PTSerif-Regular.ttf</code>

代码 17.14

然后用 `.font` 直接加载本地字体文件：

1	<code>.font {assets/fonts/NotoSerifSC-Regular.otf}</code>
2	<code>.font {assets/fonts/PTSerif-Regular.ttf}</code>
3	<code>.font {Times New Roman}</code>

代码 17.15

.font 也支持 URL 和 Google Fonts，比如：

1	<code>.font {GoogleFonts:Noto Serif SC}</code>
2	<code>.font {GoogleFonts:PT Serif}</code>
3	<code>.font {Times New Roman}</code>

代码 17.16

如果你需要更细的 class 控制，也可以用 .css 写 @font-face：

1	<code>.css</code>
2	<code>@font-face {</code>
3	<code>font-family: "MyChineseSerif";</code>
4	<code>src: url("assets/fonts/NotoSerifSC-Regular.otf") format("opentype");</code>
5	<code>}</code>
6	
7	<code>@font-face {</code>
8	<code>font-family: "MyRussianSerif";</code>
9	<code>src: url("assets/fonts/PTSerif-Regular.ttf") format("truetype");</code>
10	<code>}</code>
11	
12	<code>.font-zh {</code>
13	<code>font-family: "MyChineseSerif", serif;</code>
14	<code>}</code>
15	
16	<code>.font-ru {</code>
17	<code>font-family: "MyRussianSerif", serif;</code>
18	<code>}</code>

17.9 字体排错清单

如果你导出的 PDF 里中文或俄语显示不对，按这个顺序查：

1. 字体名是否写对。字体家族名不一定等于文件名。
2. 当前系统是否真的安装了这个字体。
3. 字体是否覆盖目标字符，比如俄文字体要支持 Cyrillic。
4. font-family fallback 里有没有可用兜底字体。
5. 如果用本地字体文件，路径是否相对 main.qd 正确。
6. 如果发布 HTML，字体文件是否会一起部署。

17.10 Quarkdown 里的字体边界

.font 适合控制全局字体家族和字号，.css 适合做语言、class、@font-face 这类精细控制。.container 支持 fontsize、fontweight、fontstyle、fontvariant 等文本样式，适合控制字号、粗细、斜体和小型大写。

例如控制字号和斜体：

这是 .container 控制字号和斜体的例子。字体家族仍然由前面的 CSS 决定。

写法：

1	.container padding:{0.4cm} border:{#94a3b8} borderstyle:{dashed} fontsize:{large} fontstyle:{italic}
2	这是 <code>.container</code> 控制字号和斜体的例子。
3	

代码 17.18

18 多文件项目

长文写到一定规模后，可以拆文件。

18.1 include

你可以把章节拆成 `chapters/intro.qd`，然后在 `main.qd` 里包含：

```
.include {chapters/intro.qd}
```

代码 18.1

拆文件的好处：

- 每章更短，更好编辑。
- 多语言版本更容易管理。
- 图片、代码、数据可以按章节组织。

18.2 推荐目录结构

1	my-book/
2	├─ main.qd
3	├─ assets/
4	└─ cover.svg
5	└─ workflow.svg
6	├─ snippets/
7	└─ hello.ts
8	└─ chapters/
9	└─ intro.qd
10	└─ layout.qd

代码 18.2

19 调试和编译

19.1 实时预览

写文档时最常用：

```
quarkdown c main.qd -p -w
```

代码 19.1

-p 打开预览，-w 监听改动。你保存文件后，浏览器会刷新。

19.2 导出 PDF

```
quarkdown c main.qd --pdf
```

代码 19.2

如果你做的是 PDF，建议用 paged 类型，并且设置页面：

1	.doctype {paged}
2	.pageformat size:{A4} margin:{2.2cm}
3	

代码 19.3

19.3 严格模式

```
quarkdown c main.qd --strict
```

代码 19.4

非严格模式下，某些错误可能显示成文档里的错误盒；严格模式会直接让命令失败，更适合做自动化检查。

19.4 清理输出目录

```
quarkdown c main.qd --clean
```

代码 19.5

注意：--clean 会清空输出目录再生成新文件，别把重要文件放到输出目录里。

20 常见写作套路

20.1 讲义模板

1	.docname {课程讲义}
2	.doctype {paged}
3	.doclang {zh}
4	.pageformat size:{A4} margin:{2cm}
5	.pagemargin {bottomcenter}
6	.currentpage / .totalpages
7	.tableofcontents
8	
9	# 第一讲
10	
11	正文。
12	

代码 20.1

20.2 技术文章模板

1	.docname {技术文章}
2	.doctype {plain}
3	.doclang {zh}
4	
5	# 问题
6	
7	## 背景
8	
9	## 实现
10	
11	## 总结

20.3 PDF 报告模板

1	.docname {项目报告}
2	.doctype {paged}
3	.theme {paperwhite} layout:{latex}
4	.numbering
5	– headings: 1.1
6	– figures: 1.1
7	– tables: 1.1
8	
9	.center
10	#! 项目报告
11	
12	.pagebreak
13	.tableofcontents
14	.pagebreak
15	
16	# 摘要
17	
18	# 正文
19	

代码 20.3

21 函数速查

类别	函数	说明
元信息	.docname	文档名
元信息	.docauthors / .docauthor	作者
元信息	.doctype	文档类型
样式	.font	全局字体和字号
样式	.css	注入 CSS 美化表格、代码框、盒子
页面	.pageformat	页面尺寸、边距、栏数
页面	.pagemargin	页眉页脚和边注
页面	.currentpage / .totalpages	当前页和总页数
页面	.pagebreak	手动换页
目录	.tableofcontents	自动目录
编号	.numbering	控制章节、图表、公式编号
引用	.ref	交叉引用
布局	.center / .align	对齐
布局	.row / .column / .grid	横排、纵排、网格
布局	.float	浮动元素
布局	.container	容器、字号、粗细、斜体
组件	.box	提示框、警告框
代码	.code	增强代码块
图表	.mermaid	Mermaid 图
数据	.read	读取文件
脚本	.var	变量

函数速查

脚本	.if	条件判断
----	-----	------

表 21.1: Quarkdown 函数速查表

表 [表 21.1](#) 可以当作你以后写 Quarkdown 时的第一页小抄。

22 结语

Quarkdown 最好的学习方式不是先背完所有函数，而是先写一篇真实文档：封面、目录、图片、表格、公式、页码，这些一旦跑通，你就已经能完成大多数 PDF 和技术长文。

下一步可以做三件事：

1. 把这份 main.qd 复制成自己的模板。
2. 用 `quarkdown c main.qd -p -w` 边改边看。
3. 改出一份你自己的讲义、论文笔记或项目说明书。

Original references: [Quarkdown Wiki](#) · [Quarkdown Docs](#)